

8 Monte Carlo Methods

8.1 Introduction and comparison to Numerical Methods

In many statistical problems we are interested in finding the expectation of a random variable. In Bayesian inference, we need the expectation of the parameter under the posterior. In frequentist methods, we need the Fisher information, which is the expectation of the second derivative of the log likelihood.

Expectation is an integration. We shall look at general methods of finding integrals. There exist methods for doing this numerically. Suppose $g : D \rightarrow \mathbb{R}$ be a function on $D \subseteq \mathbb{R}^n$ and we are interested in evaluating

$$\int_D g(x) dx. \quad (1)$$

When $n = 1$, then this can be achieved by deterministic methods like quadrature to achieve errors of rate $O(m^{-k})$ where m is the number of subintervals and k will depend on the method and other parameters. For example for Simpson's rule with 3 nodes $k = 3$. This procedure entails evaluation of the function at mk points. There are two drawbacks of this procedure

1. Although the order of the error is known, we do not have any bounds on the error as that will depend on the smoothness of the function and can be arbitrarily large.
2. The procedure gets less efficient when we go to higher dimensions. For the same error rate in n dimensions, we shall have to evaluate the function at $m^n k$ points. If we evaluate at the same mk number of points, the error rate becomes $m^{-k/n}$.

Monte Carlo or simulation methods overcome both of these drawbacks. Firstly, we can give (at least approximate) bounds on the actual error and not only the rates. Secondly, the error rate does not change with the dimension and is always $\sqrt{m}$.

Suppose we are interested in evaluating the integral in equation (1). Let $X_1, \dots, X_m$ be iid uniform random vectors on D . By the law of large numbers, with probability one

$$g(\bar{X})_m = \frac{1}{m}(g(X_1) + \dots + g(X_m)) \rightarrow E(g(X_1)) = \frac{\int_D g(x) dx}{\text{Area}(D)}$$

Also, from the central limit theorem,

$$\sqrt{m} \frac{g(\bar{X})_m - E(g(X_1))}{\sqrt{\text{Var}(g(X_1))}} \Rightarrow \mathcal{N}(0, 1)$$

Using $g(\bar{X})_m \text{Area}(D)$ as an approximation for $\int_D g(x) dx$ is the simplest Monte Carlo method for evaluating the integral and it achieves an error rate of $\sqrt{M}$

m	$g(X)_m$
1	0.774
10	0.872
100	0.834
1000	0.833
10^4	0.840
10^5	0.841

Table 1: Table 8.1

irrespective of the dimension of D . A bound can be put on the error using the cut-off points of the normal distribution which is reasonably precise for large values of m .

Example We want to integrate $g(x) = \cos(x)$ for x between zero and one. We can do this analytically to get $\sin(1) = 0.841471$. If we do this by Monte Carlo method, we generate $X_1, \dots, X_m$ as independent uniform[0,1] random variables and compute $g(\bar{X})_m$. The values we get for different values of m are in Table 8.1.

The central block of the method described above is generation of random variables X_i . We shall discuss the method of generation of uniform random variables in section 8.2. In section 8.3 we shall describe some methods of generating random variables from other distributions and the need to do so. Note that for the hypotheses of LLN and CLT to hold, we need a sequence of independent random realizations from the distribution of interest.

8.2 Generating sequence of Uniform[0,1] random numbers

In this section we shall discuss some algorithms for generating a sequence $U_1, \dots, U_m$ of m uniform (0,1) random numbers. An important thing to note is that since these numbers are generated by an algorithm, or rule, they are not really random. Such observations are called pseudo-random. The aim is to generate a sequence of numbers between zero and one that has statistical properties of a random sequence. In particular we need to check that

1. Each observation is uniformly distributed in [0,1]
2. The observations are mutually independent

8.2.1 Linear Congruent Generator

A linear congruential generator is a recurrence of the following form:

$$\begin{aligned} x_{i+1} &= ax_i \bmod m \\ u_{i+1} &= x_{i+1}/m \end{aligned}$$

Here, the multiplier a and the modulus m are integer constants that determine the values generated, given an initial value (seed) x_0 . The seed is an integer

between 1 and $m - 1$ and is generally specified by the user. Because the result of the mod m operation is always an integer between 0 and $m - 1$, the output values u_i are always between 0 and $(m - 1)/m$. Value of x_0 should not be zero, as in that case, all subsequent values of x will be zero. Here is an R code to run the generator.

```
m=17
a=6
x=7
u=rep(0,20)
for (i in 1:20){
  x=(a*x)%m
  u[i]=x/m
}
```

With this specification, the values of x are

8, 14, 16, 11, 15, 5, 13, 10, 9, 3, 1, 6, 2, 12, 4, 7, 8, 14, 16, 11

and the values of u_i are $x_i/17$. Note that the first 16 elements are distinct. The 17th element is same as the first and then the sequence repeats itself. Choosing large values of m is recommended as that will create larger loops. Since the procedure is deterministic, loops are inevitable. With the same m and x_0 , if we change a to 2, then the values of x are

14, 11, 5, 10, 3, 6, 12, 7, 14, 11, 5, 10, 3, 6, 12, 7, 14, 11, 5, 10

Here the length of the loop is 8 and all integers between 1 and 17 do not appear in the sequence. Depending on the starting value x_0 , we get one of two possible disjoint sets. This is also true for $a = 8, 9$ or 15. With $a = 4$ or 13 the loop is of length 4. With $a = 16$, the loop is of length 2.

Thus we see that with the same value of m we can have loops of different length, depending on the corresponding value of a . We would, of course like to have loops as long as possible, as once the loop ends, the series just repeats itself and is therefore deterministic. The first condition for that is to choose a large enough m . The second condition is to choose an appropriate value of a . A linear congruential generator that produces all $m - 1$ distinct values before repeating is said to have full period. When m is prime, full period is achieved from any seed $x_0 \neq 0$ if

1. $a^{m-1} - 1$ is a multiple of m
2. $a^j - 1$ is not a multiple of m for $j = 1, \dots, m - 2$.

A number a satisfying these two properties is called a primitive root of m . In the example, with $m = 17$ and $a = 16$, we have $a^2 - 1 = 255$ is a multiple of m . So the second condition is not satisfied and we have a shorter loop. The length of the loop is the smallest integer j such that $a^j - 1$ is a multiple of

Modulus (m)	Multiplier(a)
2147483647	39373
2147483399	40692
2147483563	40014

Table 2: Table 8.2

m . Table 8.2 displays moduli and multipliers for several linear congruential generators that have been recommended in the literature. A long cycle length ensures that the values of the random variable are densely distributed in the interval $(0, 1)$. The next concern is the relation between multiple generated observations. Ideally we would like them to be independent. Starting with pairs of consecutive realizations, $(U_i, U_{i+1}), i = 1, 2, \dots$ should be uniformly distributed over the unit square. With $m = 17$ and $a = 6$ (note this gives a full cycle), the plot of (U_i, U_{i+1}) is shown in figure 8.2.1. Note that the points lie in a grid. This phenomenon is characteristic of all linear congruential generators.

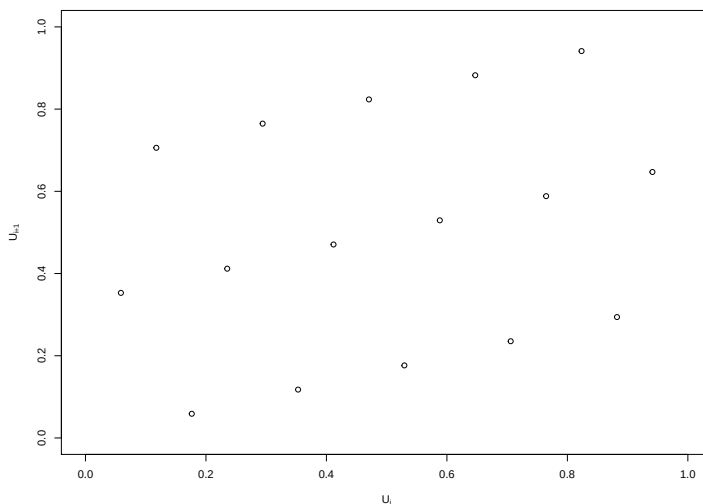


Figure 1: Plot of consecutive realizations of random numbers generated by the linear congruent generator with $m = 17$ and $a = 6$.

Of course this is not desirable as this contradicts the independence condition. The lattice structure of linear congruential generators is often used to compare their outputs and select parameters. The less pronounced the lattice structure, the more desirable is a particular generator. Obviously it is not possible to completely eliminate this lattice structure as there is an inherent dependence between successive observations due to the generating mechanism.

8.2.2 Combining Generators

We have seen that the linear congruent generator, although quite simple in execution suffers from two main drawbacks. Firstly the length of the period may not be large enough and if we need millions of random numbers there will inevitable be repetition of the full sequence. Secondly there is a clear lattice structure that conflicts with the assumption of independence. The lattice structure also means that the points they are supposed to be randomly distributed on a two-dimensional square really lie on a few straight lines, that is finite union of one-dimensional subspaces. So most of the area inside the square remains unexplored. One way to move beyond the basic linear congruential generator is to combine multiple generators through. A combination of generators can have a much longer period than any of its components.

The following combining algorithm is proposed by Wichmann and Hill (1982). Start with J pairs of values $a_j, m_j, j = 1, \dots, J$ and define J sequences of linear congruent generators $u^{(1)}, \dots, u^{(J)}$ as follows

$$\begin{aligned} x_{j,i+1} &= a_j x_{j,i} \bmod m_j, \\ u_{i+1}^{(j)} &= x_{j,i+1} / m_j, j = 1, \dots, J. \end{aligned} \quad (2)$$

Finally set u_{i+1} equal to the fractional part of $\sum_{j=1}^J u_{i+1}^{(j)}$.

Another way to combine generators is suggested by L'ecuyer(1988) where it is shown that the combination does not exhibit lattice structure at a magnification where each of the individual sequences does. The procedure is to first generate multiple sequences as in (2), then generate an intermediate sequence as

$$z_{i+1} = \sum_{j=1}^J (-1)^{(j-1)} x_{j,i+1} \bmod (m-1)$$

where m is maximum of $(m_1, \dots, m_J)$. Finally, the output u is given by

$$u_{i+1} = \begin{cases} z_{i+1}/m_1, & z_{i+1} > 0 \\ (m-1)/m, & z_{i+1} = 0 \end{cases}$$

8.3 General Sampling Methods

Let us suppose we have a satisfactory method for generating a long independent sequence of uniform(0,1) random numbers. The next question of interest is to generate a sequence of random numbers from a different specified distribution. This could be either a well-known or a new distribution, depending on the problem.

Using the R Generator R has a large number of functions that will generate random samples from standard distributions. If built-in function in R is available then we can use it directly. However, if the built-in functions are NOT available then we have to use generic methods to draw samples from such distributions.

First we discuss some general methods that work for all distributions.

8.3.1 Inverse transform

Suppose we want to sample from a cumulative distribution function F , that is, we want to generate a random variable X with the property that $P(X \leq x) = F(x) \forall x$. Define the function F^{-1} as

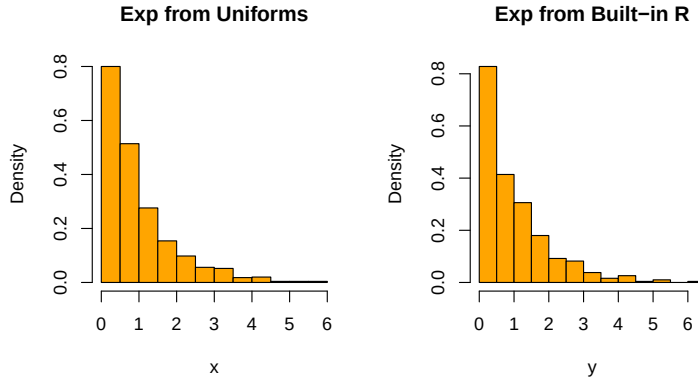
$$F^{-1}(u) = \inf t : F(t) \geq u \quad \text{for } 0 < u < 1$$

It follows that $F(x) \geq u \iff F^{-1}(u) \leq x$. Suppose $U \sim U(0, 1)$. Define $X = F^{-1}(U)$. Then $X \sim F$ since $P(X \leq x) = P(F^{-1}(U) \leq x) = P(F(x) \geq U) = F(x)$. We can use this to generate random variables from the distribution F , by first generating a uniform(0,1) random variable and then applying the function F^{-1} .

Let us now consider two special cases, namely F is strictly continuous and F is purely discrete. When F is continuous, then given u , there is a unique value t such that $F(t) = u$. Thus the function F^{-1} is a mathematical inverse.

Let us take the example of a geometric distribution. Here $F(x) = 1 - \exp(-\lambda x)$. We put $u = F(x)$ and solve for x in terms of u to get $x = -\lambda^{-1} \log(1 - u)$.

Following R code generate sample from the `Exponential(λ=1)`. We simulate samples generated by inverse transform method in `x` and samples `y` generated by built-in function `rexp` in R.



We compare their distributional behaviour by plotting the histogram and apply Kolmogorov-Smirnov test to check if the samples indeed from same distributions?

The null-hypothesis of the Kolmogorov-Smirnov test is two samples `x` and `y` are from the same distributions. The **p-value** of the test is larger than 0.05. Hence we cannot reject the null-hypothesis. It means the two sample are indeed from the same distributions.

This inverse transform method is useful for other probability distributions; ones obtained as a transformation of uniform random variables

CDF of Logistic distribution:

$$F(x) = \frac{1}{1 + \exp\left(-\frac{x-\mu}{\lambda}\right)}$$

CDF of Cauchy distribution:

$$F(x) = \frac{1}{2} + \frac{1}{\pi} \arctan((x - \mu)/\sigma)$$

If $X_i \stackrel{iid}{\sim} Exp(1)$; three standard distribution can be derived as

- $Y = 2 \sum_{i=1}^n X_i \sim \chi_{2n}^2$
- $Y = \beta \sum_{i=1}^n X_i \sim Gamma(n, \beta)$
- $Y = \frac{\sum_{i=1}^{n_1} X_i}{\sum_{i=1}^{n_1+n_2} X_i} \sim Beta(n_1, n_2)$

where $n \in \mathcal{N} = \{1, 2, \dots\}$. These transformations are quite simple and we will use them quite often. However, there is a limit to their usefulness. Only when closed form CDF is available the method is available. The method will not work for Gaussian distribution as the CDF is not in closed form.

When F is discrete, X takes values $x_1 < \dots < x_k$ with probabilities $p_1, \dots, p_k$ such that $\sum_{i=1}^k p_i = 1$. $F(x) = \sum_{i=1}^l p_i$ where $l = \sup m : x_m \leq x$. Thus $F^{-1}(u) = x_l \iff \sum_{i=1}^{l-1} p_i < u \leq \sum_{i=1}^l p_i$. We generate a sample of size 1000 from a discrete distribution with support 1, 2, 3, 4 with probability vector p and draws a side-by-side barplot of the actual and observed probabilities in figure 8.3.1.

8.4 Generating random vectors from Multinomial

The cdf of a multivariate distribution is a function from $\mathbb{R}^k$ to $[0, 1]$. and is not invertible. There is no equivalent of inversion theorem here.

Suppose we are interested in generating an observation from Multinomial($n, \underline{p}$). One way to do this is to generate n independent random vectors, each Multinomial($1, \underline{p}$) and take their sum. To generate each of these, generate u from uniform $(0, 1)$. Then find i such that

$$\sum_{i=1}^{l-1} p_i < u \leq \sum_{i=1}^l p_i.$$

The generated random vector has 1 in the i -th position and 0 in all others.

When n is large, this is very inefficient. Note that for a multinomial $X = (X_1, \dots, X_k)$, the marginal distribution of X_1 is binomial(n, p_1) and the conditional distribution of X_j given $X_1, \dots, X_{j-1}$ is

$$binomial(n - X_1 - \dots - X_{j-1}, p_j / (p_j + \dots + p_k))$$

We can use the inversion method to generate each of these.

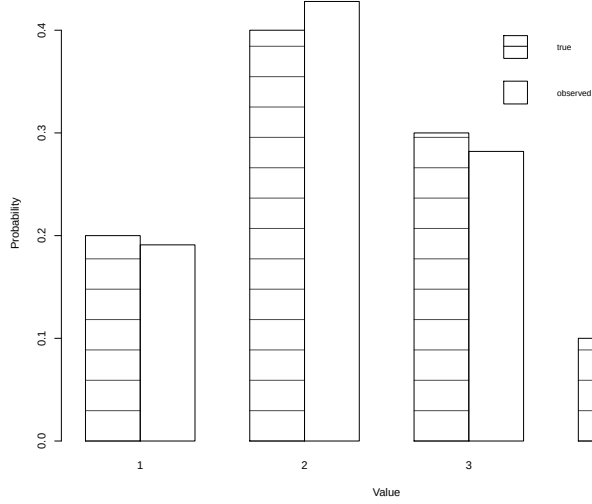


Figure 2: Barplot of actual and observed probabilities for a sample of size 1000 from a discrete distribution.

8.5 Generating Normal Random Variables

Note that if $Z \sim \mathcal{N}(0, 1)$, then $X = \sigma Z + \mu \sim \mathcal{N}(\mu, \sigma^2)$. Hence, it suffices to consider sampling from $\mathcal{N}(0, 1)$. We assume the availability of a sequence $U_1, U_2, \dots$ of independent random variables uniformly distributed on the unit interval $[0, 1]$ and consider methods for transforming these to normally distributed random variables.

8.5.1 Inverse Transform Method

The general inverse transform method can be used to generate samples from the normal distribution. Since the inverse of the normal cdf is not available in closed form, it is necessary to obtain numerical approximations. Even the cdf is not available in closed form and needs to be evaluated. Evaluation of the cumulative normal is in itself necessary for many financial applications (including evaluation of the Black- Scholes formula). One way is to use numerical integration methods. Here we describe an alternative method. It takes the form

$$\Phi(x) \approx 1 - \phi(x)(b_1 t + b_2 t^2 + b_3 t^3 + b_4 t^4 + b_5 t^5), \quad t = \frac{1}{1 + px},$$

for constants b_i and p for positive x , where ϕ is the normal pdf and Φ is the normal cdf. The approximation extends to negative arguments through the identity $\Phi(-x) = 1 - \Phi(x)$. This method has a maximum absolute error less than 7.5×10^{-8} . The necessary constants for this approximation are as follows:

$$\begin{array}{lll} b_1 = 0.319381530 & b_2 = -0.356563782 & b_3 = 1.781477937 \\ b_4 = -1.821255978 & b_5 = 1.330274429 & p = 0.2316419 \end{array}$$

The problem of computing the inverse $\Phi^{-1}(u)$ can be posed as one of finding the root x of the equation $\Phi(x) = u$ and in principle addressed through any general root-finding algorithm. Newton's method, for example, produces the iterates

$$x_{n+1} = x_n - \frac{\Phi(x_n) - u}{\phi(x_n)} \quad (3)$$

For this, at every step of iteration, one needs to evaluate $\Phi(x_n)$ numerically as described above.

8.5.2 Box-Muller Method

It is clear from the above discussion that the inversion method involves numerous computations and approximations. This requires high computational time and leads to accumulation of errors. A simple alternative method is the Box-Muller algorithm. This algorithm generates a pair of independent univariate standard normal random variables at every step. The algorithm is based on the following two properties of the bivariate normal. If $Z \sim \mathcal{N}_2(0, I)$, then

1. $R = Z_1^2 + Z_2^2$ is exponentially distributed with mean 2, i.e., $P(R \leq x) = 1 - e^{-x/2}$
2. Given R , the point (Z_1, Z_2) is uniformly distributed on the circle of radius $\sqrt{R}$ centered at the origin.

Thus, to generate (Z_1, Z_2) , we may first generate R and then choose a point uniformly on the circle of radius $\sqrt{R}$. To sample from the exponential distribution we may set $R = -2 \log(U_1)$, with $U_1 \sim \text{Unif}[0, 1]$. To generate a random point on a circle, we may generate a random angle uniformly between 0 and 2π and then map the angle to a point on the circle. The random angle may be generated as $V = 2\pi U_2$, $U_2 \sim \text{Unif}[0, 1]$. The corresponding point on the circle has coordinates $(\sqrt{R} \cos(V), \sqrt{R} \sin(V))$. The complete algorithm is given as follows:

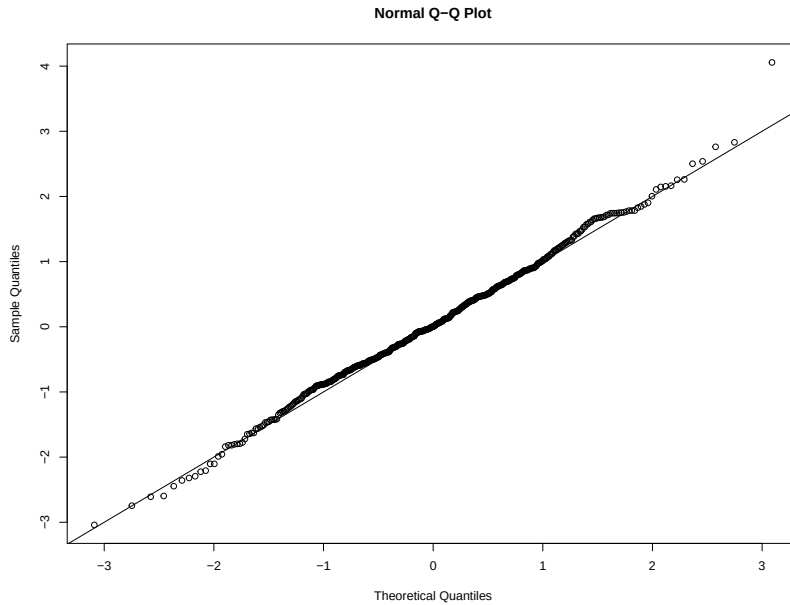
- generate U_1, U_2 independent $\text{Unif}[0, 1]$
- $R = -2 \log(U_1)$
- $V = 2\pi U_2$
- $Z_1 = \sqrt{R} \cos(V)$
- $Z_2 = \sqrt{R} \sin(V)$

Marsaglia developed a modification of the Box-Muller method that reduces computing time by avoiding evaluation of the sine and cosine functions. Instead, the Marsaglia-Bray method uses acceptance-rejection to sample points uniformly in the unit disc and then transforms these points to normal variables.

The R command to generate normal random variable is `rnorm`. The method used can be "Kinderman-Ramage", "Buggy Kinderman-Ramage" (not for `set.seed`), "Ahrens-Dieter", "Box-Muller", "Inversion", "user-supplied". The default is `Inversion`. The command `RNGkind()[2]` can be used to find out which method is being used. The default method can be changed by resetting it as `RNGkind(normal.kind = "Box-Muller")`. For the actual inversion, the program `qnorm` provides precise results up to about 16 digits.

We generate 1000 replications of normal random samples of size 500 and tests for normality for each of them using the Shapiro-Wilk test. Finally it reports what proportion of the p-values are less than 5%. We repeat the process after changing the method to Box-Muller. In both cases we get the observed proportion to be slightly over 5% indicating that the generated samples can be considered to be from normal distribution. The qq-plot of one such sample is presented in figure 3. The points lie roughly on the straight line with zero intercept and unit slope, as expected from a normal sample.

Figure 3: Q-Q plot of a sample of size 500 from normal distribution



8.6 Generating Multivariate Normal

A d dimensional random vector follows multivariate normal distribution if all linear combinations follow univariate normal. A multivariate normal distribution $\mathcal{N}_d(\mu, \Sigma)$ is specified by its mean vector μ and covariance matrix Σ . Note

that if $Z \sim \mathcal{N}_d(0, I)$ and $X = \mu + AZ$, then $X \sim \mathcal{N}_d(\mu, AA^T)$. Using any of the methods discussed above, we can generate independent standard normal random variables $Z_1, \dots, Z_d$ and assemble them into a vector $Z \sim \mathcal{N}_d(0, I)$. Thus, the problem of sampling X from the multivariate normal $\mathcal{N}_d(\mu, \Sigma)$ reduces to finding a matrix A for which $AA^T = \Sigma$. Two common methods of achieving this are described below. Each has its own advantages. We shall return to these methods for the simulation of Brownian motion.

Cholesky Factorization Among all such A s for which $AA^T = \Sigma$, a lower triangular one is particularly convenient because it reduces the calculation of $\mu + AZ$ to the following:

$$\begin{aligned} X_1 &= \mu_1 + A_{11}Z_1 \\ X_2 &= \mu_2 + A_{21}Z_1 + A_{22}Z_2 \\ &\dots \\ X_d &= \mu_d + A_{d1}Z_1 + A_{d2}Z_2 + \dots + A_{dd}Z_d. \end{aligned}$$

A full multiplication of the vector Z by the matrix A would require approximately twice as many multiplications and additions.

A representation of Σ as AA^T with A lower triangular is a Cholesky factorization of Σ . Considering a $d \times d$ covariance matrix Σ , we need to solve

$$\begin{pmatrix} A_{11} & & & \\ A_{21} & A_{22} & & \\ \vdots & \vdots & \ddots & \\ A_{d1} & A_{d2} & \dots & A_{dd} \end{pmatrix} \begin{pmatrix} A_{11} & A_{21} & \dots & A_{d1} \\ & A_{22} & \dots & A_{d2} \\ & & \ddots & \vdots \\ & & & A_{dd} \end{pmatrix} = \Sigma. \quad (4)$$

Traversing the Σ_{ij} by looping over $j = 1, \dots, d$ and then $i = j, \dots, d$ produces a set of equations. Exactly one new entry of the A matrix appears in each equation. Thus we can solve for the individual entries sequentially to obtain the Cholesky factorization explicitly as follows:

$$\begin{aligned} A_{ij} &= (\Sigma_{ij} - \sum_{k=1}^{j-1} A_{ik}A_{jk})/A_{jj}, \quad j < i, \\ A_{ii} &= \sqrt{\Sigma_{ii} - \sum_{k=1}^{i-1} A_{ik}^2}. \end{aligned}$$

Eigenvector Factorization and Principal Components The equation $AA^T = \Sigma$ can also be solved by diagonalizing Σ . As a symmetric $d \times d$ matrix, Σ has d real eigenvalues $\lambda_1, \dots, \lambda_d$, and because Σ must be positive definite or semidefinite the λ_i are nonnegative. Furthermore, Σ has an associated orthonormal set of eigenvectors $v_1, \dots, v_d$. It follows that $\Sigma = V\Lambda V^T$, where V is the orthogonal matrix with columns $v_1, \dots, v_d$ and Λ is the diagonal matrix with diagonal entries $\lambda_1, \dots, \lambda_d$. Hence, if we choose $A = V\Lambda^{1/2}$ then $AA^T = V\Lambda V^T = \Sigma$.

Unlike the Cholesky factor, the matrix A in has no particular structure providing a computational advantage in evaluating AZ , nor is this matrix faster to compute than the Cholesky factorization. The eigenvectors and eigenvalues of a covariance matrix do however have a statistical interpretation that is occasionally useful. We discuss this interpretation next.

If Σ has full rank, then A must have full rank and $X = AZ$ implies $Z = BX$ with $B = A^{-1}$. Thus, the factors Z_j are themselves linear combinations of the X_i . Since, we have $A^{-1} = \Lambda^{-1/2}V^T$, it follows that Z_j is proportional to $v_j^T X$. The factors Z_j are optimal in a precise sense. Suppose the eigenvalues are ordered in decreasing order of magnitude and the Z_j 's are ordered accordingly. The best single-factor approximation to X ; i.e., the linear combination $w^T X$ that best captures the variability of the components of X is Z_1 . The next best factor orthogonal to the first is Z_2 and so on. In the statistical literature, the linear combinations $v_j^T X$ are called the principal components of X . We may thus say that the principal components provide an optimal lower-dimensional approximation to a random vector. The variance explained by the first k principal components is the ratio

$$\frac{\lambda_1 + \lambda_2 + \cdots + \lambda_k}{\lambda_1 + \lambda_2 + \cdots + \lambda_k + \cdots + \lambda_d}$$

The R function to generate multivariate normal is `mvrnorm` inside the `MASS` package. The matrix decomposition is done via principal components method. If $n = 1$ a vector of the same length as μ is returned. Otherwise an n by $\text{length}(\mu)$ matrix with one sample in each row is returned. The function `rmvnorm` in the package `mvtnorm` allows changing the method to Cholesky. Alternatively, one can directly compute the Cholesky factorization of Σ as in the code that generates $N = 100$ replicates of bivariate normal with specified mean and variance.